



Research paper

powdR: An R package for quantitative mineralogy using full pattern summation of X-ray powder diffraction data

Benjamin M. Butler^{a,*}, Stephen Hillier^{a,b}^a The James Hutton Institute, Craigiebuckler, Aberdeen AB15 8QH, UK^b Department of Soil and Environment, Swedish University of Agricultural Sciences (SLU), SE-75007, Uppsala, Sweden

ARTICLE INFO

Dataset link: <https://github.com/benmbutler/powdR>, <https://CRAN.R-project.org/package=powdR>

Keywords:

Quantitative mineralogy
X-ray powder diffraction
Full pattern summation
Full pattern fitting
RockJock

ABSTRACT

X-ray powder diffraction (XRPD) is consistently found to be the most accurate analytical technique for qualitative and quantitative mineralogy. For environmental samples such as rocks, sediments and soils that can contain a range of crystalline, disordered and amorphous components, quantitative XRPD methods implemented in Excel-based programmes such as FULLPAT and RockJock have proved particularly appropriate. Such programmes quantify phase concentrations in complex mixtures by treating the diffractogram of a given mixture as the sum of contributions from individual components. Thus, using this full pattern summation (FPS) approach, an observed diffractogram of a sample can be modelled by scaling the contributions of the diffractograms for all pure phases within it and then phase concentrations computed using reference intensity ratios.

Here the principles implemented in FULLPAT, RockJock and an in-house application for FPS have together been used to create the powdR package for R. The powdR package advances on previous implementations of FPS in terms of functionality, speed (parallel computation) and adaptability (via use of bespoke R code), and includes a Shiny web application for user-friendly analysis. powdR can be used to quantify samples prepared either with or without an internal standard, and includes the public domain RockJock library of 169 pure diffractograms that cover a wide range of phases that may be encountered in soil and rock samples. Furthermore, powdR contains functionality for automated phase quantification, which can select appropriate phases from a comprehensive reference library (e.g., RockJock). Here the FPS functions implemented in powdR and their use are described in detail.

1. Introduction

X-ray powder diffraction (XRPD) is an analytical technique widely applied in the study of mineral mixtures. A typical XRPD measurement results in a diffractogram (or ‘pattern’) comprised of discrete ‘Bragg’ diffraction peaks varying in intensity (y), expressed for example in counts, along an experimental axis (x) usually expressed in degrees 2θ . The ‘Bragg’ peaks from crystalline mineral phases rise above a background, which may also include diffuse scattering for example from X-ray amorphous phases. Mineral identification from this data is achieved by comparing the peak positions and their relative intensities to diffraction data from pure phases, often using databases such as the Powder Diffraction File (Gates-Rector and Blanton, 2019). In addition to qualitative identification, and provided sample preparation is appropriate, the intensities of crystalline (e.g., quartz, calcite and gypsum), disordered (e.g., clay minerals), and amorphous (e.g., volcanic glass and organic matter) signals within a diffractogram can be related to

their concentrations within the mixture (Chipera and Bish, 2002). This principal facilitates the quantification of phase concentrations from XRPD data.

Results from round-robin competitions for quantification of complex mixtures illustrate the versatility of XRPD for quantitative analysis even in cases where crystalline, disordered and amorphous phases are present within the same sample (Raven and Self, 2017; Butler and Hillier, in press). Of the approaches available for quantitative XRPD, the simple Reference Intensity Ratio (RIR) method has consistently proven as accurate as more sophisticated Rietveld methods (Omotoso et al., 2006; Chipera and Bish, 2013). A RIR is a measure of the diffracting power of a phase relative to that of a standard (most often corundum, Al_2O_3), usually measured in a 50:50 mixture by weight (Hillier, 2000). The RIR of a detectable phase within a mixture is required for its quantification (Hillier, 2003).

Historically, the RIR method was initially applied to individual peaks (Hillier, 2000), but full pattern approaches like those pioneered

* Corresponding author.

E-mail address: benjamin.butler@hutton.ac.uk (B.M. Butler).

by Smith et al. (1987) have since proven superior (Chipera and Bish, 2002; Eberl, 2003; Omotoso et al., 2006). Using this full pattern methodology, an observed diffractogram is treated as the sum of contributions from all phases within the mixture (i.e., full pattern summation; FPS). Thus, using prior measured patterns of pure phases (hereafter 'reference patterns'), an observed pattern can be modelled by scaling reference pattern intensities until the fitted pattern closely matches that of the observed. The final step in this process uses the RIRs to compute phase concentrations.

Current FPS approaches are mainly implemented in Microsoft Excel spreadsheets using the Solver™ add-in, with examples including FULLPAT (Chipera and Bish, 2002), RockJock (Eberl, 2003), and our in-house unpublished XRPDBULK [referred to as 'X-LS-MINERAL' in Kleeborg, 2005]. Whilst versatile and accurate, implementation in Excel does not facilitate user friendly functionality particularly in the sense of tidy data [where each variable is a column, each observation is a row, and each type of observational unit is a table; Wickham et al., 2014]. For high-throughput datasets, Excel-based implementations can also prove difficult to modify for parallel computing. Lastly, a key component of FPS is often expert interpretation of the observed and fitted patterns along with the associated residuals, which can be challenging with the limited graphical capabilities of Excel. For these reasons we present an implementation of FPS as a package for the R Language and Environment for Statistical Computing (R Core Team, 2020).

Implementing FPS in R as the package powdR implies that the software is open source and multi-platform. Here we introduce the methodological background of quantitative FPS (Section 2) before describing the main functionality of the powdR package and its novel features using reproducible examples (Sections 3 and 4). These novel features include: (1) the implicit ability to harness the powerful data analysis capabilities of R and its extensions; (2) automatic harmonisation of the sample and library onto the same 2θ scale (Section 3.2.3); (3) advanced alignment routines for enhanced accuracy (Sections 3.2.7 and 3.2.9); (4) an algorithm for automated FPS (Section 3.3); (5) simple computation of batches of samples (Sections 4.1 and 4.2); (6) interactive plotting of summation results (Section 4.4); (7) the ability to run batches using parallel processing (Section 4.5); and (8) the ability to run functions via either code or a Shiny web application (Section 4.6).

2. Methodological background

Fig. 1 illustrates how an observed diffractogram of a mixture can be considered as the sum of all of the pure components within it. Thus, when appropriate reference patterns are available, they can be used to accurately model the observed pattern. Fitting is achieved by computation of a scaling coefficient for each reference pattern, s , by which it is multiplied to achieve an optimised fit (Fig. 1).

Once scaling coefficients are derived, quantitative estimates of phase concentrations can be calculated using the RIRs (Srodon et al., 2001; Hillier, 2003). The method for calculation of phase concentrations using RIRs depends on whether or not the sample was prepared with a known concentration of internal standard. If the sample does not contain an internal standard, one must assume that all detectable phases within the mixture can be identified with appropriate reference patterns, and their concentrations sum to 100 weight %. Thus in absence of an internal standard, the concentration (weight %) of component i (X_i) can be obtained via:

$$X_i = \frac{s_i / \text{RIR}_i}{\sum s / \text{RIR}} \times 100 \quad (1)$$

where s_i and RIR_i in the numerator denote the scaling coefficient and RIR of component i , respectively. In the denominator, s and RIR represent vectors of scaling coefficients and reference intensity ratios covering all phases in the sample, respectively.

Alternatively, if the sample contains an internal standard of known concentration, then the concentration of a given phase can be obtained via:

$$X_i = \frac{X_{std}}{\text{RIR}_i / \text{RIR}_{std}} \times \frac{s_i}{s_{std}} \quad (2)$$

where X_i , RIR_i and s_i are as above, whilst X_{std} , RIR_{std} and s_{std} are the known concentration (weight percent), RIR and scaling coefficient of the internal standard, respectively.

The computationally intensive aspect of phase quantification via FPS is the derivation of s for each phase in the sample, which is most often achieved by least squares optimisation (Chipera and Bish, 2002). For the computation of scaling coefficients powdR implements several options, including the minimisation of an objective function and non-negative least squares.

2.1. Objective functions and optimisation algorithms

In determining scaling coefficients for all phases during FPS, the user seeks to minimise the difference between the observed and fitted patterns. As outlined in Chipera and Bish (2002), several objective functions that summarise this difference can be minimised during FPS. powdR implements three functions, *Delta*, *R* and *R_{wp}*, defined as:

$$\text{Delta} = \sum |I_m - I_c| \quad (3)$$

$$R = \left(\frac{\sum (I_m - I_c)^2}{\sum (I_m)^2} \right)^{\frac{1}{2}} \quad (4)$$

$$R_{wp} = \left(\frac{\sum [(I_m)^{-1} \times (I_m - I_c)^2]}{\sum [(I_m)^{-1} \times (I_m)^2]} \right)^{\frac{1}{2}} \quad (5)$$

where I_m and I_c are vectors of measured and calculated intensities, and $| \cdot |$ denotes absolute values. Eq. (3) is from Chipera and Bish (2002) whilst Eqs. (4) and (5) are detailed for example in Howard and Preston (1989).

Four optimisation routines can be selected in powdR for minimisation of *Delta*, *R* or *R_{wp}*, all of which are implemented in base R (R Core Team, 2020). These include the variable metric algorithm of Broyden (1970), Fletcher (1970), Goldfarb (1970) and Shanno (1970), hereafter 'BFGS'; the downhill simplex method of Nelder and Mead (1965), hereafter 'Nelder-Mead'; and the conjugate gradients method of Fletcher and Reeves (1964), hereafter 'CG' [note that the acronyms for each algorithm reflect those used in R; R Core Team, 2020].

2.2. Non-negative least squares

As a fast alternative to minimisation of an objective function, it is also possible to derive scaling coefficients in powdR using non-negative least squares (hereafter NNLS), which constrains coefficients to a minimum of zero. Functionality for NNLS in R is provided in the nnls package (Mullen and van Stokkum, 2012), which is based on the FORTRAN code of Lawson and Hanson (1995). Whilst NNLS is fast, when used in FPS of XRPD data, the results are often slightly inferior to that of minimising the objective functions outlined above (see Supplementary Material). For this reason NNLS is mainly included in powdR as a method for fast identification of phases likely present within an observed pattern, rather than quantitative phase analysis.

2.3. Limits of detection

The limit of detection (LOD) for a component within a sample is related to its RIR and the composition of the sample. The LOD is inversely proportional to the RIR, thus stronger diffractors (such as quartz with an RIR, for example, of 5.4) are usually detectable in lower concentrations than weaker diffractors (such as illite with an RIR, for example, of 0.3).

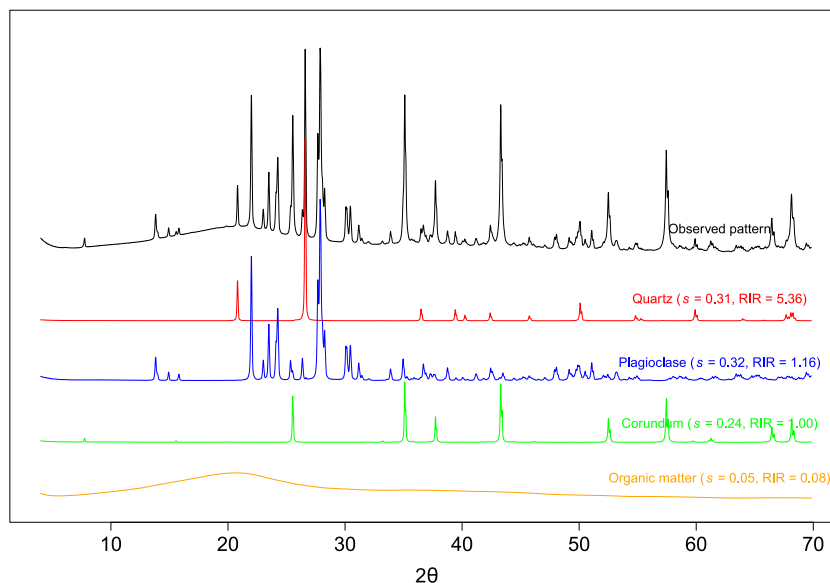


Fig. 1. An example illustrating how a theoretical observed pattern is comprised of the sum of scaled patterns of the pure components within it (quartz, plagioclase, corundum and organic matter). In this case the pure patterns have each been scaled (from identical maximum intensities; Section 3.1.1) via their respective scaling coefficients (s) so that their sum exactly matches the observed pattern.

If a sensible LOD estimate for a given phase within the sample can be supplied, then the LOD of any other phase within a sample may also be estimated using the RIRs via the following equation adapted from Hillier (2003):

$$\text{LOD}_x = \text{LOD}_y \times \frac{\text{RIR}_y}{\text{RIR}_x} \quad (6)$$

where x and y denote the phase for which the LOD is to be determined, and the phase for which the LOD has been estimated, respectively. This principle may be used to identify phases within a FPS output that may be below their respective LOD and, thus potentially omit them from the analysis. Note that the RIRs in Eq. (6) must be normalised relative to a common standard (see Section 3.1.1).

3. Design and functionality of powdR

3.1. Reference libraries

A key component of FPS analysis is the library of reference patterns. In the case of powdR, a reference library contains diffractograms of pure phases, each scaled to the same maximum (but arbitrary) intensity, and each with a corresponding normalised RIR (see Section 3.1.1). To ensure the reference patterns are provided in the right format for subsequent functions, powdR implements a constructor function for reference libraries, `powdRlib()`. The `powdRlib()` function requires two tables of information. First is a data frame of all pure reference patterns for the library and their 2θ scale, provided as the `xrd_table` argument (e.g., Table 1). Second, provided as the `phases_table` argument, is a data frame that defines the IDs, names and RIRs of each phase provided in the `xrd_table` argument (e.g., Table 2). Supplying both an ID and a name for each phase allows the user to provide multiple patterns for the same phase (that may differ in detail), which can then be grouped together in the final quantitative output depending on the level of detail required. For example, 'Quartz_a' and 'Quartz_b' in Table 2 can be grouped together by the `phase_name` column to summarise the total concentration of quartz.

Upon supplying the `xrd_table` and `phases_table` to `powdRlib()`, the data are converted into an S3 (R Core Team, 2020) `powdRlib` object, which is a list of three components. The first component, named `xrd`, is a data frame of the count intensities of each reference pattern provided in the `xrd_table` argument, with each

Table 1

The basic form required for the `xrd_table` argument of the `powdRlib()` function. The first column, 'tth', represents the 2θ scale common to all reference patterns in the library. Each subsequent column title specifies the unique phase ID of each reference pattern.

tth	Quartz_a	Quartz_b	Albite	Illite_2M1	Obsidian
3.00	4726	4157	3788	4706	7335
3.05	5001	4322	1800	4125	7975
3.10	4587	3880	2927	4235	6764
3.15	5091	4192	3306	4082	7575
3.20	4447	3996	2729	4284	7175
3.25	4414	3874	2417	3852	6613
3.30	3827	4107	2838	4752	6652
3.35	3838	3617	2469	3342	6172
3.40	4077	3786	2326	3985	6346
...	...	...	...	...	...

Table 2

An example of the basic form required for the `phases_table` argument of the `powdRlib()` function, with each phase relating to those defined in the `xrd_table` argument (e.g. Table 1). Whilst the `phase_id` must be unique for each phase in the library, the `phase_name` column allows for phases to be grouped when more than one reference pattern is available for a given phase.

phase_id	phase_name	rir
Quartz_a	Quartz	4.62
Quartz_b	Quartz	4.34
Albite	Plagioclase	1.31
Illite_2M1	Illite	0.22
Obsidian	Obsidian	0.14

column representing a different reference pattern. The second component, named `tth`, is a vector of the 2θ scale of the reference patterns also provided in the `xrd_table` argument. The third component, named `phases`, is a dataframe of the phase ID, name and RIR provided in the `phases_table` argument, with each row specifying these details for a different reference pattern. The column-wise order of the `xrd` data frame of a `powdRlib` object matches the row-wise order of the `phases` data frame of the same object.

3.1.1. The RockJock reference library

Supplementary to custom `powdRlib` objects that can be created by the user, the `powdR` package also comes pre-loaded with the public

domain RockJock reference library of Eberl (2003). The RockJock library in powdR contains 169 pure (in absence of an internal standard) reference patterns of various crystalline, disordered and amorphous phases commonly encountered in soil and other geological samples. The method of sample preparation is provided in Eberl (2003) and is based on that of Srodon et al. (2001). In brief, each reference pattern was analysed from 5 to 65° 2 θ using Cu K α on a Siemens D500 diffractometer, and RIRs measured by preparation of standards with ZnO (Eberl, 2003; Srodon et al., 2001).

In the original RockJock library, the count intensities of each reference pattern are that of the original measurement, from which RIRs were computed (Eberl, 2003). However, in the RockJock library included in powdR, each of the reference patterns has been scaled to the same arbitrary maximum intensity (10,000 counts) and the RIRs normalised relative to corundum. Normalisation of RIRs to corundum is recommended because it allows comparison with other corundum-normalised RIRs (Hillier, 2003), such as those in the Powder Diffraction File [I/I_{cor}; Gates-Rector and Blanton, 2019], and it is also a requirement to have RIRs on a common scale when estimating LODs using Eq. (6) (Section 2.3).

To achieve corundum-normalisation of the RockJock library, first the ratio of the maximum intensity of the corundum reference pattern (16,715) to its integrated intensity (provided in RockJock and computed between 19 and 64.5° 2 θ , yielding 19,384) was calculated and multiplied by its RIR (1), yielding 0.86234. The same calculation was done for each of the remaining reference patterns in the RockJock library, and each derived value divided by that calculated for corundum (i.e., 0.86234) to yield its corundum-normalised RIR. This calculation of normalised RIRs is simply a convenience to express the RIRs relative to corundum, which is the convention that has been used at The James Hutton Institute in the unpublished 'XRPDBULK' programme since 2004 (Kleeberg, 2005; Hillier, 2015, 2018).

3.2. Full pattern summation

Full pattern summation is implemented in powdR as the `fps()` function. The steps implemented in `fps()` are illustrated in Fig. 2. The function accepts up to fifteen arguments, most of which are optional. Summaries of these arguments are provided in Table 3 and more detailed descriptions given in Sections Section 3.2.1 to 3.2.11. The resulting output from `fps()` is a `powdRfps` object (of the S3 object oriented system in R), comprised of a list of ten components that are summarised in Table 4.

3.2.1. lib

The `lib` argument is used to specify the library of pure reference patterns. Data provided to the `lib` argument must be a `powdRlib` object that can be derived from the constructor function outlined in Section 3.1.

3.2.2. smpl

The `smpl` argument requires the sample XRPD pattern as a two column data frame. The first column must define the 2 θ axis and the second column the count intensities. It is worth noting here that the `rxylib` package (Kreutzer and Johannes Friedrich, 2017) can be used to load various proprietary XRPD file formats into R. Further, the free software `PowDLL` (Kourkoumelis, 2013) can be particularly useful for converting proprietary files into ASCII format such as `.xy` or `.csv`.

3.2.3. harmonise

For cases where the 2 θ ranges/resolutions of the reference library and sample differ (but are measured using the same monochromatic wavelength) the `harmonise` argument specifies whether to force the data onto the same 2 θ scale. If `harmonise` = TRUE (the default), then the data will be subset to the overlapping 2 θ range before conversion onto the 2 θ scale of the coarsest data using cubic spline interpolation.

This novel feature of powdR acts to ensure seamless use of data when samples and reference libraries come from different instruments or perhaps the same instrument with an altered 2 θ range or step size. It is important to note that the harmonisation functionality in powdR does not bypass the fact that the most accurate results from FPS are obtained when the samples and standards are measured on the same or similar instrument with the same X-ray source and instrumental parameters. Whilst meaningful quantitative results can be obtained using a reference library measured on a different instrument to the sample [e.g., Butler et al., 2020], visual inspection of the results will always be key to assessing whether the fitted pattern adequately represents the observed pattern.

3.2.4. solver and obj

The `solver` argument is used to specify which optimisation algorithm (Sections 2.1 and 2.2) is used. The argument accepts one of 'BFGS', 'Nelder-Mead', 'CG' or 'NNLS'. If either 'BFGS', 'Nelder-Mead' or 'CG' are used as the solver argument, then an objective function has to be minimised. The choice of objective function (Section 2.1) is specified using the `obj` argument, which accepts one of 'Delta' (Eq. (3)), 'R' (Eq. (4)) or 'Rwp' (Eq. (5)). Preliminary testing of all possible combinations that can be provided to the `solver` and `obj` arguments (see Supplementary Material) found the combination of 'BFGS' and 'Rwp' to provide the best performance in terms of computation time and accuracy, and these are hence implemented as the default values (Table 3).

3.2.5. refs

The `refs` argument is used to specify which reference patterns from the library to use during FPS. The argument requires a string of phase IDs or names that must match those found in the `lib$phases$phase_id` or `lib$phases$phase_name` columns.

3.2.6. std, std_conc and normalise

The `std` argument is used to specify which reference library pattern is treated as the internal standard, whilst the `std_conc` argument can be used to specify its concentration, if known (in weight %). The string provided to `std` must match a single phase ID present within the `lib$phases$phase_id` column. This internal standard can firstly be used for linear alignment of the sample (Section 3.2.7), and if a numeric value is supplied to the `std_conc` argument then it will be used in combination with Eq. (2) to quantify phase concentrations.

The `normalise` argument is a logical parameter to be used when a known internal standard concentration is supplied to the `std_conc` argument. When `normalise` = TRUE the internal standard concentration is removed from the composition and the remaining phase concentrations normalised to sum to 100 percent.

3.2.7. align, manual_align and tth_align

The `align` argument is a single numeric value used to define the maximum positive or negative shift that can be applied to the sample in order to align it to the selected standard (defined in the `std` argument). This alignment is primarily designed to correct for effects such as sample displacement that can result in small 2 θ misalignments (Butler et al., 2019). Although peak misalignment is not always linear over the full 2 θ range, a linear shift as implemented in powdR is usually a reasonable approximation.

The `manual_align` argument specifies how the value supplied to `align` is used. If `manual_align` = FALSE (the default), then the value in `align` is treated as a maximum positive or negative shift that can be applied to the sample in order to align it to the reference pattern specified in the `std` argument. Such alignment is achieved by maximising the correlation of the sample and the selected standard via a linear 2 θ shift. Alternatively, if `manual_align` = TRUE then the sample is simply shifted by the value specified by the `align` argument (which may either be positive or negative).

Table 3

Arguments used in the `fps()` and `afps()` functions along with their required format and the default value where available. Default values are applied for a given argument if it is missing from the function call. Arguments without a default must be specified in the function call.

Argument	Functions	Format	Default
<code>lib</code>	<code>fps()</code> and <code>afps()</code>	<code>powdRlib</code> object	
<code>smpl</code>	<code>fps()</code> and <code>afps()</code>	Data frame	
<code>harmonise</code>	<code>fps()</code> and <code>afps()</code>	Logical	TRUE
<code>solver</code>	<code>fps()</code> and <code>afps()</code>	Character	'BFGS'
<code>obj</code>	<code>fps()</code> and <code>afps()</code>	Character	'Rwp'
<code>refs</code>	<code>fps()</code> and <code>afps()</code>	Character	
<code>std</code>	<code>fps()</code> and <code>afps()</code>	Character	
<code>std_conc</code>	<code>fps()</code> and <code>afps()</code>	Numeric	NA
<code>normalise</code>	<code>fps()</code> and <code>afps()</code>	Logical	FALSE
<code>align</code>	<code>fps()</code> and <code>afps()</code>	Numeric	0.1
<code>manual_align</code>	<code>fps()</code> and <code>afps()</code>	Logical	FALSE
<code>tth_align</code>	<code>fps()</code> and <code>afps()</code>	Numeric	<code>c(min(smpl[[1]]), max(smpl[[1]]))</code>
<code>tth_fps</code>	<code>fps()</code> and <code>afps()</code>	Numeric	<code>c(min(smpl[[1]]), max(smpl[[1]]))</code>
<code>shift</code>	<code>fps()</code> and <code>afps()</code>	Numeric	0
<code>remove_trace</code>	<code>fps()</code>	Numeric	0
<code>amorphous</code>	<code>afps()</code>	Character	<code>c()</code>
<code>amorphous_lod</code>	<code>afps()</code>	Integer	0
<code>lod</code>	<code>afps()</code>	Numeric	0.1
<code>force</code>	<code>fps()</code> and <code>afps()</code>	Character	<code>c()</code>

The `tth_align` argument is a vector specifying the minimum and maximum of the 2θ range upon which alignment is carried out when `manual_align = FALSE`. This can be beneficial when the user wishes to focus on a specific internal standard for alignment, such as the main (most intense) quartz peak. If the `tth_align` argument is left incomplete and `manual_align = FALSE`, then optimisation of the alignment is calculated on the full 2θ range.

3.2.8. `tth_fps`

The `tth_fps` argument is a vector specifying the minimum and maximum of the 2θ range upon which FPS is carried out. This can be beneficial when the user wishes to exclude high and/or low angle data. If the `tth_fps` argument is left incomplete, the FPS is applied to the full 2θ range.

3.2.9. `shift`

Further to linear alignment of the sample to a reference pattern (Section 3.2.7), it has been found in the 'XRPDBULK' programme (Hillier, 2015, 2018) that small additional 2θ shifts applied to each reference pattern in the fitting process can yield more accurate results. Hence the `shift` argument of `fps()` and `afps()` provides an optional secondary alignment protocol whereby each reference pattern is shifted along the 2θ axis to minimise the objective function supplied in the `obj` argument. More specifically, after an initial optimisation of the scaling coefficients, the objective function is again minimised by optimising a shifting coefficient for each remaining reference pattern, which specifies its positive or negative adjustment along the 2θ axis. During the optimisation of the shifting coefficients, the scaling coefficients are fixed and reference patterns are maintained on the same 2θ scale using cubic spline interpolation. If the absolute value of any optimised shifting coefficient exceeds the value specified in the `shift` argument, its shifting coefficient is reset to zero. Shifts are then applied to the reference patterns, cubic spline interpolation used to ensure they all remain on the original 2θ axis, and the scaling coefficients re-optimised. When the `shift` argument is set to zero (the default; Table 3), no shifting is applied.

This shifting process is designed to correct for small linear differences in the peak positions of the standards relative to the sample, which may result from a combination of instrumental aberrations, mineralogical variation and/or uncorrected errors in the library patterns. Whilst it provides more accurate results, the process can substantially increase computation time.

3.2.10. `remove_trace`

In some cases an output from `fps()` may contain numerous phases with very low concentrations that are likely to be well below their limits of detection. In such cases the `remove_trace` argument specifies a limit (weight %) below which any phase is removed from the final output.

3.2.11. `force`

In order to avoid minor or trace phases being incorrectly removed from the automated analysis during LOD estimation, the `force` argument can be used. This argument takes the form of a string of IDs or names of any phases that must be retained in the final fit. Strings provided to `force` must match phase IDs and/or names present within the `lib$phases$phase_id` or `lib$phases$phase_name` columns.

3.3. Automated full pattern summation

Aside from traditional FPS where appropriate reference patterns are selected by the user, `powdR` implements a novel, automated, approach that seeks to select appropriate reference patterns from the library and subsequently remove those that may be below their limit of detection. This automated approach is implemented in `powdR` as the `afps()` function which has recently been tested on challenging clay-bearing mixtures in Butler and Hillier (in press). Many of the arguments and steps implemented in `afps()` are identical to those in `fps()`, and a flow diagram of `afps()` is provided in Fig. 3. Neither the `refs` nor `remove_trace` arguments are required in `afps()` because reference patterns are automatically selected, and then omitted using limit of detection estimates. The resulting output from `afps()` is a `powdRafps` object comprised of the same components as a `powdRfps` object (Table 4). To facilitate automation, `afps()` accepts three additional arguments to those outlined in Sections 3.2.1 to 3.2.11 (Table 3), each of which are described in more detail below.

3.3.1. `lod`

The `lod` argument defines a LOD estimate for the single phase specified in the `std` argument, from which the LOD of any other phase in the reference library can be computed using Eq. (6). The `afps()` function uses these estimates to exclude selected phases from the fitting process if they are estimated to be below their respective LOD (Fig. 3).

It must be noted that LODs can be affected by a number of factors including the sample composition (more complex phase mixtures resulting in decreased LODs), differences in adsorption characteristics, and counting statistics. It is therefore important to consider these factors when using the LOD functionality in `afps()`. As long as the

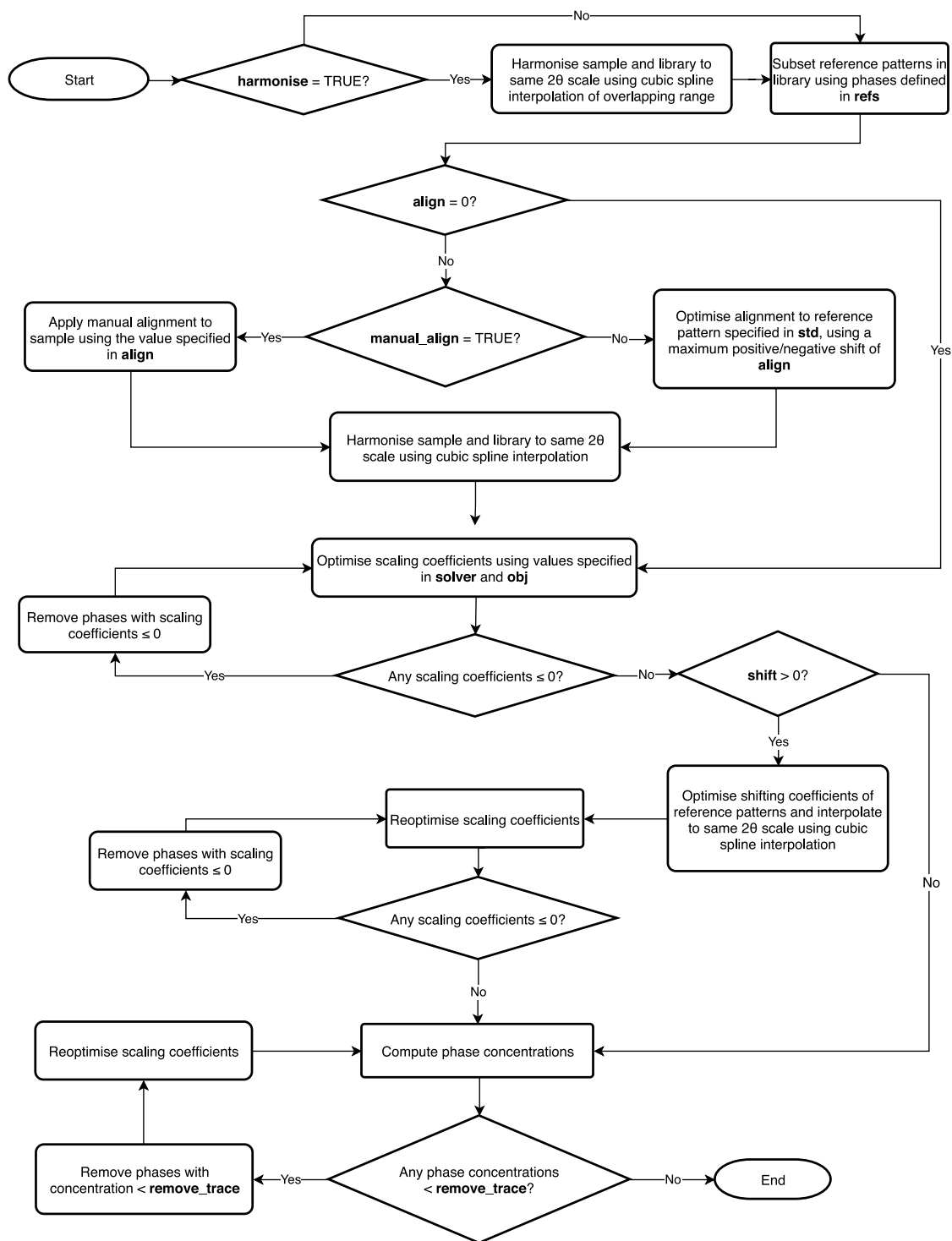


Fig. 2. Flowchart depicting the process of phase quantification using `fps()`.

value supplied to the `lod` argument of `afps()` is broadly accurate then sensible results can be obtained, as shown in [Butler and Hillier \(in press\)](#) where the same LOD assumption was applied when automatically quantifying 27 clay-bearing samples of variable composition, yielding accurate results.

3.3.2. *amorphous* and *amorphous_lod*

Quantification of amorphous phases from XRPD data can often be associated with larger uncertainties than crystalline phases, therefore

the approach used to estimate LOD's in `afps()` may result in their incorrect exclusion or inclusion. For this reason the `amorphous` and `amorphous_lod` arguments allow the user to specify which phases should be treated as amorphous (using a string of their phase IDs) and their estimated LOD, respectively (a single numeric parameter in weight %). The default of `amorphous_lod` is zero, and thus when this value is used all amorphous phases with concentrations > 0% would be retained.

Table 4
The components of `powdRfps` and `powdRafps` objects.

Component	Class	Description
<code>tth</code>	Numeric	The modified 2θ axis of data
<code>fitted</code>	Numeric	The fitted pattern on the modified 2θ axis
<code>measured</code>	Numeric	The measured pattern on the modified 2θ axis
<code>residuals</code>	Numeric	The residuals (measured — fitted) on the modified 2θ axis
<code>phases</code>	Data frame	The IDs, names, RIRs, and concentrations of quantified phases
<code>phases_grouped</code>	Data frame	The concentrations of quantified phases grouped and summed by name
<code>rwp</code>	Numeric	The R_{wp} of the fit (Eq. (5))
<code>weighted_pure_patterns</code>	Data frame	The reference patterns on the modified 2θ axis multiplied by the optimised weighting coefficients
<code>coefficients</code>	Numeric	The optimised weighting coefficients
<code>inputs</code>	List	The input arguments used to derive the output

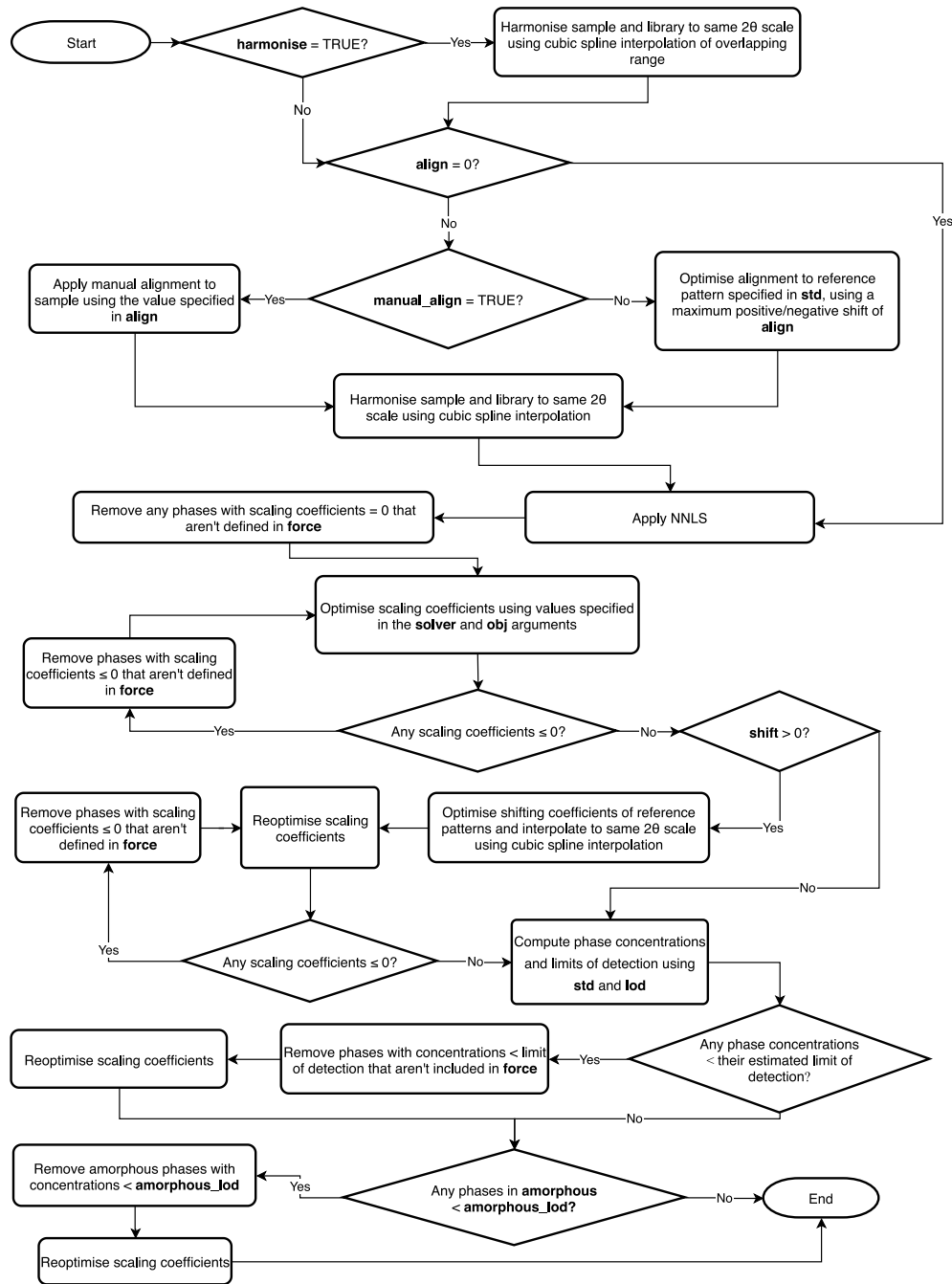


Fig. 3. Flowchart depicting the process of phase quantification using `afps()`.

4. Using powdR

Here the use and accuracy of `fps()` and `afps()` will be outlined and assessed using the `rockjock` and `rockjock_mixtures` data that are provided with `powdR`. The `rockjock` data are described in Section 3.1.1, whilst the `rockjock_mixtures` data consist of a list of eight XRPD measurements of synthetic mixtures containing various concentrations of seven mineral components: corundum (RockJock ID = 'CORUNDUM'), K-feldspar (RockJock ID = 'ORDERED_MICROCLINE'), plagioclase (RockJock ID = 'LABRADORITE'), kaolinite (RockJock ID = 'KAOLINITE_DRY_BRANCH'), dioctahedral smectite (RockJock ID = 'MONTMORILLONITE_WYO'), illite (RockJock ID = 'ILLITE_1M_RM30') and quartz (RockJock ID = 'QUARTZ').

4.1. `fps()`

A single FPS computation can be achieved via the following use of `fps()`:

```
data(rockjock_mixtures)

quant <- fps(lib = rockjock,
  smpl = rockjock_mixtures$Mix1,
  std = "CORUNDUM",
  refs = c("ORDERED_MICROCLINE",
    "LABRADORITE",
    "KAOLINITE_DRY_BRANCH",
    "MONTMORILLONITE_WYO",
    "ILLITE_1M_RM30",
    "QUARTZ",
    "CORUNDUM"),
  align = 0.3)
```

which yields a `powdRfps` object containing the results (Table 4). Since the concentration of the internal standard is not specified when calling `fps()` above, Eq. (1) is used to quantify samples, yielding sample compositions constrained to 100 weight %. Alternatively, specifying an internal standard concentration via the `std_conc` argument would yield concentrations computed using Eq. (2).

Further to running individual samples it is also possible to wrap `fps()` within `lapply()` to quantify a list of samples. In this manner, since the eight samples within the `rockjock_mixtures` data are constrained to seven of the phases present in the `rockjock` reference library, all eight samples can be quantified via:

```
multi_quant <- lapply(rockjock_mixtures, fps,
  lib = rockjock,
  std = "CORUNDUM",
  refs = c("ORDERED_MICROCLINE",
    "LABRADORITE",
    "KAOLINITE_DRY_BRANCH",
    "MONTMORILLONITE_WYO",
    "ILLITE_1M_RM30",
    "QUARTZ",
    "CORUNDUM"),
  align = 0.3)
```

which yields a list of eight `powdRfps` objects (total computation time on a single core of an Intel Core i7-6600U CPU @2.60 GHz 2.70 GHz processor = 75 s).

A comparison of the known phase concentrations to those estimated by `fps()` are provided in Fig. 4. The results illustrate the accuracy of the FPS implementation in `powdR`, with mean absolute error of known phase concentrations relative to those determined by `fps()` yielding $\pm 0.9\%$ (though it should be noted that the phases used to make the RockJock mixtures match exactly to the respective standards in the RockJock library).

4.2. `afps()`

The `afps()` function is designed for cases where the phases present within the samples to be quantified have not been identified by the user and perhaps even vary substantially on a sample-by-sample basis. In such cases, it is thus advantageous to provide a relatively large reference library (e.g., RockJock) to the dataset and for `afps()` to select appropriate phases from it.

A single automated FPS computation can be achieved via the following use of `afps()`:

```
a_quant <- afps(lib = rockjock,
  smpl = rockjock_mixtures$Mix1,
  std = "CORUNDUM",
  lod = 1,
  align = 0.3)
```

which yields a `powdRafps` object containing the results (Table 4).

The use of `afps()` is most advantageous when handling multiple samples. To illustrate this, the full `rockjock` library in `powdR` was used in combination with `afps()` to quantify phase concentrations in each of the eight synthetic mixtures present in the `rockjock_mixtures` data. Again, via the use of `lapply()`, it is relatively simple to apply `afps()` to all eight samples:

```
multi_a_quant <- lapply(rockjock_mixtures, afps,
  lib = rockjock,
  std = "CORUNDUM",
  lod = 1,
  align = 0.3)
```

which yields a list of eight `powdRafps` objects (total computation time on a single core of an Intel(R) Core(TM) i7-6600U CPU @2.60 GHz 2.70 GHz processor = 42 min). A comparison of the phase concentrations determined by `afps()` to those of `fps()` and the known values is provided in Fig. 4. The `afps` function yielded three non-identifications, all associated with low concentrations of smectite and illite (Mixes 3, 4 and 6; Fig. 4), where a phase present in the mixture was not present in the final output. Such non-identifications contribute to the mean absolute error of `afps()` relative to the known concentrations increasing to 1.8% across the `rockjock_mixtures` data. Nevertheless, this example illustrates that `afps()` can distill a large reference library down to the appropriate phases. What is sacrificed in terms of accuracy via use of `afps()` is somewhat balanced by the ability to yield relatively accurate quantitative estimates in a fraction of the time it might take to manually identify appropriate reference patterns for each sample. Further assessment of the accuracy achievable with `afps()` is provided in Butler and Hillier (in press).

Manual inspection of outputs from `afps()` and `fps()` would always be encouraged and should be considered an essential part of the process of obtaining accurate results via identification of phases that have either not been accounted for or are inappropriate for the sample. In this way, the `afps()` approach can at the very least act to speed up analysis of large datasets by doing a large part of the work for the user, before expert interpretation of the outputs can fine tune the results for enhanced accuracy.

4.3. Summarising quantitative data

When deriving lists of `powdRfps` or `powdRafps` objects for quantification of multiple samples (e.g., Sections 4.1 and 4.2), it can often be useful to summarise the estimates of phase concentrations in a single table, which in `powdR` can be achieved using the `summarise_mineralogy()` function. For example,

```
summary_table <- summarise_mineralogy(multi_quant,
  type = "grouped",
  order = TRUE,
  rwp = TRUE)
```

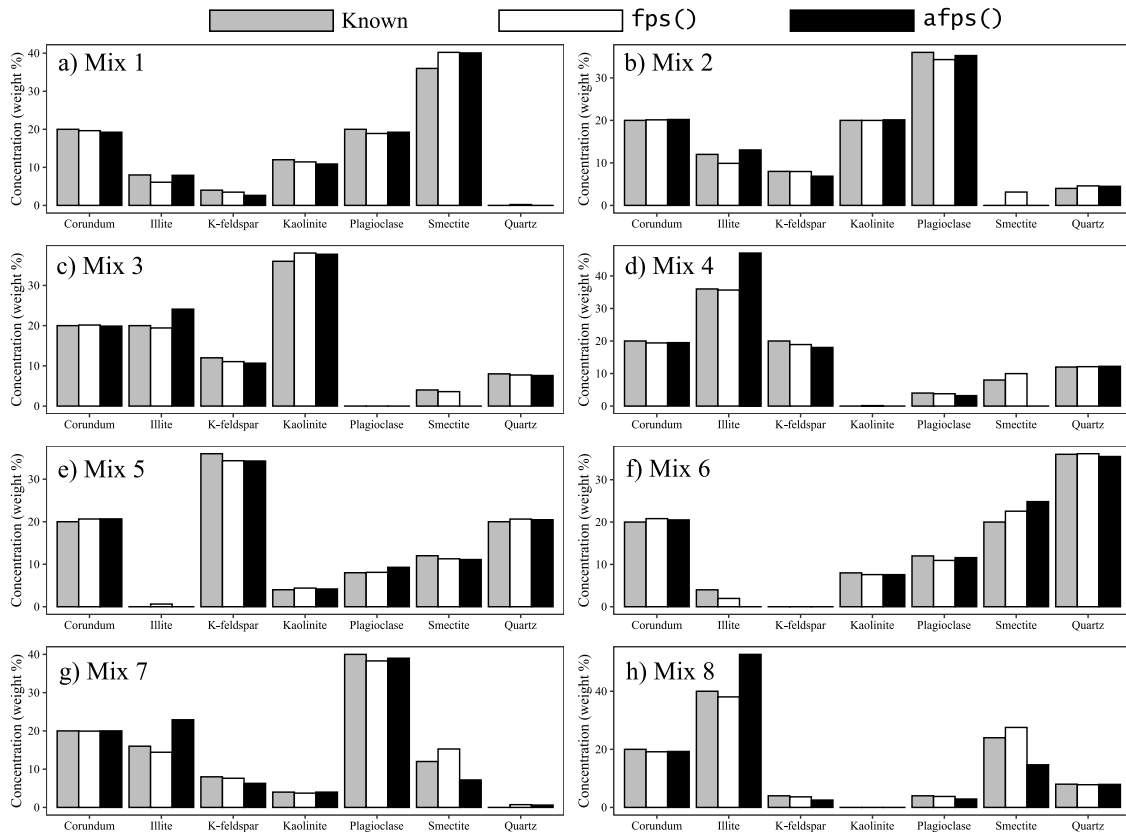



Fig. 4. Comparison of known concentrations of the `rockjock_mixtures` data to estimates provided by `fps()` and `afps()`. Note that the slightly higher errors associated with clay minerals (illite, kaolinite and smectite) align with the more challenging nature of their quantification due to the broad peaks associated with their disordered structures (Raven and Self, 2017; Butler and Hillier, in press).

yields the summary table provided in Table 5. In the above example, `multi_quant` is a list of eight `powdRfps` objects derived in Section 4.1, `mode = "grouped"` specifies that the reference patterns are grouped according to their phase name, `order = TRUE` specifies that phases are ordered according to the sum of their occurrence across the samples, and `rwf = TRUE` specifies that the R_{wfp} (Eq. (5)) is included as the final column as an objective measure of the quality of the fit.

4.4. Plotting

Given the importance of visual inspection of results when applying `fps()` and `afps()`, `powdR` contains functionality designed to make plotting the data as simple as possible. Any `powdRfps` or `powdRafps` object can be plotted using `plot()`.

More specifically, for a given `powdRfps` or `powdRafps` object, `x`, a plot can be obtained via:

```
plot(x, wavelength = 1.5406, mode = "both")
```

where the `wavelength` argument is required for computation of d-spacings using Bragg's law (in units of Angstroms, Å), and `mode = "both"` designates that both the fit and residuals are plotted. The above code produces a plot of the format displayed in Fig. 5, where the measured, fitted and scaled reference patterns are provided on the top and the residuals provided below. For easier interpretation of the data, adding `interactive = TRUE` to the function call is recommended, which creates an html plot that can be loaded in RStudio or an internet browser. Interactive features of these html plots allow the user to zoom and pan the data, view the d-spacings and toggle the various patterns on or off, therefore allowing for improved visualisation of results compared to Excel-based FPS implementations.

4.5. Parallel processing

An additional advantage of the `powdR` package over other implementations of the FPS approach is that it can be used in combination with other R packages for additional functionality. When combined with the `foreach` (Microsoft and Weston, 2017) and `doParallel` packages (Microsoft and Weston, 2018), the computation of `fps()` or `afps()` can be spread across multiple cores of a given machine to yield much improved processing times, for example:

```
library(foreach)
library(doParallel)

#Detect number of cores on machine
UseCores <- detectCores()

#Register the cluster using n - 1 cores
cl <- makeCluster(UseCores-1)
registerDoParallel(cl)

#Use foreach loop and %dopar% to compute in parallel
aquant_parallel <- foreach(i=1:length(rockjock_mixtures)) %dopar%
  (powdR::afps(lib = rockjock,
    smpl = rockjock_mixtures[[i]],
    std = "CORUNDUM",
    lod = 1,
    align = 0.3))

#name the items in the aquant_parallel list
names(aquant_parallel) <- names(rockjock_mixtures)

#stop the cluster
```

Table 5

An example summary table derived from a list of eight `powdRfps` objects using `summarise_mineralogy()`. NA values represent phases that yielded negative weighting coefficients during computation via `fps()` (Fig. 2) and were therefore omitted from quantification.

sample_id	Corundum	Smectite (Di)	Illite	Plagioclase	Quartz	K-feldspar	Kaolinite	rwf
Mix1	19.6	40.2	6.1	18.9	0.2	3.5	11.4	0.119
Mix2	20.1	3.1	9.9	34.3	4.6	8.0	20.0	0.124
Mix3	20.1	3.6	19.4	NA	7.7	11.1	38.1	0.114
Mix4	19.4	10.0	35.7	3.8	12.1	18.9	0.1	0.137
Mix5	20.6	11.3	0.6	8.1	20.6	34.4	4.4	0.122
Mix6	20.8	22.6	2.0	10.9	36.1	NA	7.6	0.142
Mix7	19.9	15.3	14.4	38.3	0.7	7.6	3.7	0.128
Mix8	19.2	27.5	38.1	3.8	7.8	3.6	NA	0.124

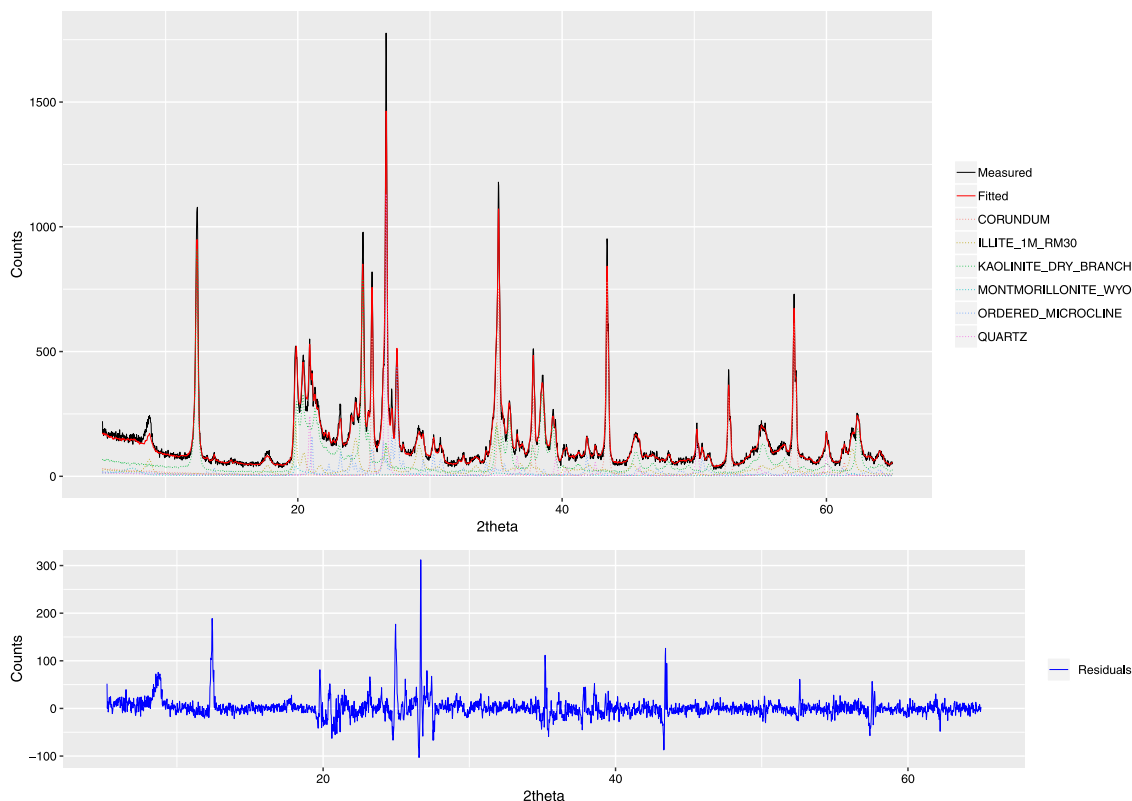


Fig. 5. An example plot of a `powdRfps` or `powdRafps` object. Solid black and red lines (top) are the measured and fitted patterns, respectively, whilst the dotted lines are the weighted contributions of various reference patterns contributing to the fit. The solid blue line (bottom) represents the residuals, calculated as the measured pattern minus the fitted pattern. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

```
stopCluster(cl)
```

decreases the computation time for the eight samples from 42 min (when only one core of an Intel(R) Core(TM) i7-6600U CPU @2.60 GHz 2.70 GHz processor is used) to 18 min (when three cores of the same processor are used). Such computation can easily be scaled up to compute in parallel on any number of cores where available.

4.6. The *powdR* Shiny application

As a user-friendly option for running *powdR* that does not require R code, the package also contains a Shiny (Chang et al., 2018) web application that provides a graphical user interface for much of its functionality. This web application is loaded in the user's default web browser using `run_powdR()` (Fig. 6). In *powdR* version 1.2.4 the web application has six tabs: 1) Reference Library Builder for creating `powdRlib` objects; 2) Reference Library Viewer for plotting the reference patterns in a given `powdRlib` object; 3) Reference Library editor for easy subsetting of `powdRlib` objects; 4) Full Pattern Summation for quantifying individual samples using `fps()` or `afps()`; 5) Results Viewer/Editor for

plotting, tabulating and editing outputs from `fps()` and `afps()`; and 6) Help, which provides a video tutorial (hosted on YouTube) for each of the previous five tabs.

5. Final remarks

The main functionality within the *powdR* package has been introduced along with detailed descriptions of how the `fps()` and `afps()` functions are implemented and the associated results summarised/visualised. The implementation of FPS in *powdR* includes a range of novel features (Section 1) along with the implicit ability to harness the powerful data manipulation and analysis capabilities of the R language and its extensions. Unlike other available implementations of FPS, *powdR* can be run using either bespoke R code or via a graphical user interface provided as a Shiny web application. With reproducible examples provided here and within the package documentation, in combination with the widely known RockJock library included with the package, it is anticipated that the functionality of *powdR* will allow for improved and efficient FPS analysis of randomly oriented soil, sediment and rock samples analysed by XRPD.

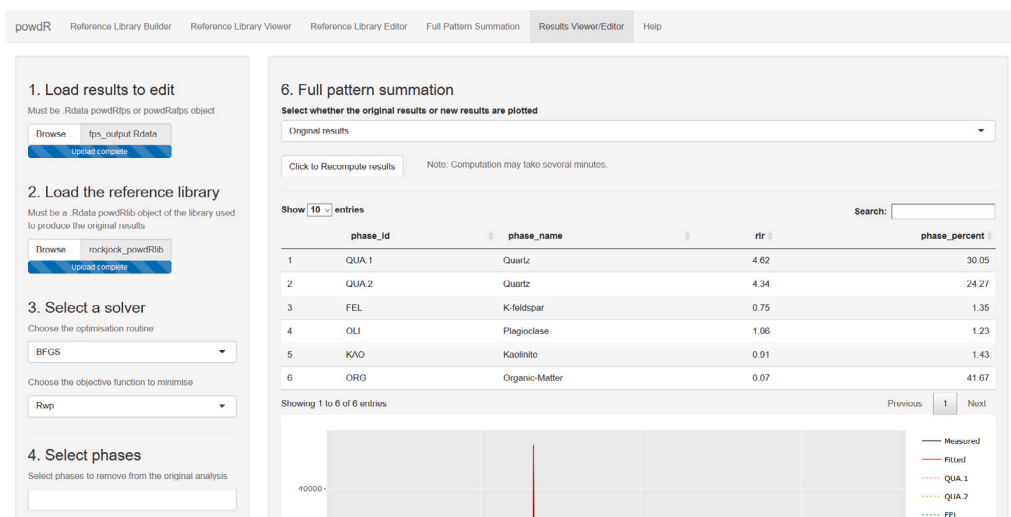


Fig. 6. Screenshot of the graphical user interface of the powdR Shiny application.

Computer code availability

powdR version 1.2.4 and archived sources are available under the GNU General Public Licence from the Comprehensive R Archive Network [<https://cran.r-project.org/package=powdR>; Butler and Hillier (2020)], and the development version available on GitHub (<https://github.com/benmbutler/powdR>).

CRedit authorship contribution statement

Benjamin M. Butler: Conceptualisation, Software development, Formal analysis, Writing - original draft. **Stephen Hillier:** Conceptualisation, Methodology development, Supervision, Writing - review & editing.

Data availability

Links to open source code and data:

- <https://github.com/benmbutler/powdR>
- <https://CRAN.R-project.org/package=powdR>

Acknowledgements

This work was supported by a Macaulay Development Trust Fellowship, United Kingdom, Grant No. MDT-50. The support of the Scottish Government's Rural and Environment Science and Analytical Services Division (RESAS), UK is also gratefully acknowledged. We are particularly grateful to Denis Eberl for providing permission to include the RockJock data in powdR, which offers a valuable and comprehensive reference library for others to use. powdR was initially inspired by functionality within the aqp package for R, authored by Dylan Beaudette, and we are thankful for Dylan's help and support during the early stages of powdR's development. Lastly we thank Helen Pendlowski for reviewing the manuscript prior to submission, and also thank Tyler Kane, Kate Campbell and Elizabeth Rampe for their constructive and insightful reviews that helped improve this paper.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cageo.2020.104662>.

References

- Broyden, C.G., 1970. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA J. Appl. Math.* 6 (1), 76–90.
- Butler, B., Hillier, S., 2020. powdR: Full pattern summation of X-ray powder diffraction data. R package version 1.2.4. URL <https://CRAN.R-project.org/package=powdR>.
- Butler, B.M., Hillier, S., 2021. Automated full pattern summation of X-ray powder diffraction data for high-throughput quantification of clay-bearing mixtures. *Clay Clay Miner. CLAY-D-20-00007*. in press.
- Butler, B.M., Palarea-Albaladejo, J., Shepherd, K.D., Nyambura, K.M., Towett, E.K., Sila, A.M., Hillier, S., 2020. Mineral–nutrient relationships in African soils assessed using cluster analysis of X-ray powder diffraction patterns and compositional methods. *Geoderma* 375, 114474.
- Butler, B.M., Sila, A.M., Shepherd, K.D., Nyambura, M., Gilmore, C.J., Kourkoumelis, N., Hillier, S., 2019. Pre-treatment of soil X-ray powder diffraction data for cluster analysis. *Geoderma* 337, 413–424.
- Chang, W., Cheng, J., Allaire, J., Xie, Y., McPherson, J., 2018. shiny: Web application framework for R. R package version 1.2.0. URL <https://CRAN.R-project.org/package=shiny>.
- Chipera, S.J., Bish, D.L., 2002. FULLPAT: A full-pattern quantitative analysis program for X-ray powder diffraction using measured and calculated patterns. *J. Appl. Crystallogr.* 35 (6), 744–749.
- Chipera, S.J., Bish, D.L., 2013. Fitting full X-ray diffraction patterns for quantitative analysis: A method for readily quantifying crystalline and disordered phases. *Adv. Mater. Phys. Chem.* 03 (01), 47–53.
- Eberl, D., 2003. User Guide to ROCKJOCK - A Program for Determining Quantitative Mineralogy from X-Ray Diffraction Data. Tech. Rep., US Geological Survey.
- Fletcher, R., 1970. A new approach to variable metric algorithms. *Comput. J.* 13 (3), 317–322.
- Fletcher, R., Reeves, C.M., 1964. Function minimization by conjugate gradients. *Comput. J.* 7 (2), 149–154.
- Gates-Rector, S., Blanton, T., 2019. The powder diffraction file: a quality materials characterization database. *Powder Diff.* 34 (4), 352–360.
- Goldfarb, D., 1970. A family of variable-metric methods derived by variational means. *Math. Comput.* 24 (109), 23–26.
- Hillier, S., 2000. Accurate quantitative analysis of clay and other minerals in sandstones by XRD: comparison of a Rietveld and a reference intensity ratio (RIR) method and the importance of sample preparation. *Clay Miner.* 35, 291–302.
- Hillier, S., 2003. Quantitative analysis of clay and other minerals in sandstones by X-Ray powder diffraction (XRPD). In: *Clay Mineral Cements in Sandstones*. International Association of Sedimentologists, pp. 213–251.
- Hillier, S., 2015. X-ray powder diffraction full-pattern summation methods for quantitative analysis of clay bearing samples. In: *Euroclay 2015 Programme and Abstracts*. p. 174.
- Hillier, S., 2018. Quantitative analysis of clay minerals and poorly ordered phases by prior determined X-ray diffraction full pattern fitting: procedures and prospects. In: *9th Mid-European Clay Conference Book*, p. 6.
- Howard, S., Preston, K., 1989. Profile fitting of powder diffraction patterns. *Mod. Powder Diff.* 20, 217–275.
- Kleeberg, R., 2005. Results of the second Reynolds Cup contest in quantitative mineral analysis. In: *Commission on Powder Diffraction Newsletter*, Vol. 30. International Union of Crystallography, pp. 22–24.

- Kourkouvelis, N., 2013. PowDLL: a reusable .NET component for interconverting powder diffraction data. Recent developments. In: Lisa O'Neill (Ed.), ICCDD Annual Spring Meetings, p. 28.
- Kreutzer, S., Johannes Friedrich, 2017. rxylib: Import XY-data into R. R package version 0.2.1. URL <https://CRAN.R-project.org/package=rxylib>.
- Lawson, C.L., Hanson, R.J., 1995. Solving Least Squares Problems, Vol. 15. SIAM.
- Microsoft, Weston, S., 2017. foreach: Provides foreach looping construct for R. R package version 1.4.4. URL <https://CRAN.R-project.org/package=foreach>.
- Microsoft, Weston, S., 2018. doParallel: Foreach parallel adaptor for the 'parallel' package. R package version 1.0.14. URL <https://CRAN.R-project.org/package=doParallel>.
- Mullen, K.M., van Stokkum, I.H.M., 2012. nnls: The lawson-hanson algorithm for non-negative least squares (NNLS). R package version 1.4. URL <https://CRAN.R-project.org/package=nnls>.
- Nelder, J.A., Mead, R., 1965. A simplex method for function minimization. *Comput. J.* 7 (4), 308–313.
- Omotoso, O., McCarty, D.K., Hillier, S., Kleeberg, R., 2006. Some successful approaches to quantitative mineral analysis as revealed by the 3rd Reynolds Cup contest. *Clays Clay Miner.* 54 (6), 748–760.
- R Core Team, 2020. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria, URL <https://www.R-project.org/>.
- Raven, M.D., Self, P.G., 2017. Outcomes of 12 years of the Reynolds Cup quantitative mineral analysis round robin. *Clays Clay Miner.* 65 (2), 122–134.
- Shanno, D.F., 1970. Conditioning of quasi-Newton methods for function minimization. *Math. Comput.* 24 (111), 647–656.
- Smith, D.K., Johnson, G.G., Scheible, A., Wims, A.M., Johnson, J.L., Ullmann, G., 1987. Quantitative X-ray powder diffraction method using the full diffraction pattern. *Powder Diff.* 2 (2), 73–77.
- Srodon, J., Drits, V.A., McCarty, D.K., Hsieh, J.C., Eberl, D.D., 2001. Quantitative X-ray diffraction analysis of clay-bearing rocks from random preparations. *Clays Clay Miner.* 49 (6), 514–528.
- Wickham, H., et al., 2014. Tidy data. *J. Stat. Softw.* 59 (10), 1–23.